

ĐẠI HỌC QUỐC GIA TP.HCM  
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

LỚP: CS112.Q11.KHTN

MÔN HỌC: PHÂN TÍCH VÀ THIẾT KẾ THUẬT TOÁN

---

Báo Cáo Bài Tập Nhóm 1

---

Nhóm 4:  
Lê Văn Thức  
Bảo Quý Định Tân

Giảng viên:  
Nguyễn Thanh Sơn

# 1 Giới thiệu bài toán

Chúng ta có hai mảng số nguyên  $A$  (độ dài  $n$ ) và  $B$  (độ dài  $m$ ). Hai người chơi là Alice và Bob tham gia một trò chơi, với Alice đi trước. Trò chơi diễn ra theo lượt. Ở lượt thứ  $i$  (bắt đầu từ 1), người chơi hiện tại phải thực hiện thao tác:

- Chọn một chỉ số  $k$  trong mảng  $A$  sao cho  $A[k] = B[i]$ .
- Xóa toàn bộ hậu tố của mảng  $A$  từ vị trí  $k$  đến cuối mảng (tức là xóa các phần tử  $A[k], A[k+1], \dots, A[\text{end}]$ ). Mảng  $A$  sau đó chỉ còn lại các phần tử từ đầu đến  $k-1$ .

Nếu đến lượt của một người chơi mà họ không thể thực hiện thao tác hợp lệ (không tìm thấy  $A[k] = B[i]$  trong phần còn lại của mảng  $A$ ), người đó sẽ thua cuộc. Nhiệm vụ là xác định người chiến thắng giả định cả hai chơi tối ưu.

## 2 Phương pháp thiết kế thuật toán

### 2.1 Phương pháp sử dụng: Tham lam (Greedy)

Để giải quyết bài toán này, chúng ta sử dụng phương pháp thiết kế thuật toán Tham lam để mô phỏng nước đi tối ưu của hai người chơi.

**Cơ chế hoạt động:** Mục tiêu của mỗi người chơi là chiến thắng, tức là ép đối phương rơi vào thế không còn nước đi hợp lệ.

- Tại mỗi lượt thứ  $i$ , người chơi cần tìm giá trị  $B[i]$  trong mảng  $A$ .
- Sau khi chọn vị trí  $k$ , mảng  $A$  bị cắt ngắn, chỉ giữ lại phần tiền tố  $0 \dots k-1$ .
- Để gây khó khăn tối đa cho đối thủ ở lượt tiếp theo ( $i+1$ ), người chơi hiện tại cần để lại phần tiền tố ngắn nhất có thể. Tiền tố càng ngắn, xác suất đối thủ tìm thấy  $B[i+1]$  càng thấp.

Do đó, chiến thuật tối ưu cho cả Alice và Bob là: Luôn chọn vị trí xuất hiện đầu tiên (nhỏ nhất) của  $B[i]$  trong mảng  $A$ .

### 2.2 Chi tiết giải thuật

1. **Tiền xử lý:** Duyệt mảng  $A$  để lưu trữ vị trí xuất hiện đầu tiên ('first\_pos') của từng giá trị. Nếu một số xuất hiện nhiều lần, ta chỉ quan tâm đến chỉ số nhỏ nhất vì theo chiến thuật tham lam, đây là lựa chọn tốt nhất.
2. **Khởi tạo:** Đặt 'current\_limit' là giới hạn trên của chỉ số có thể chọn. Ban đầu 'current\_limit = n + 1' (hoặc vô cực).
3. **Mô phỏng lượt chơi:** Duyệt lần lượt các phần tử 'val' trong mảng  $B$ :
  - Kiểm tra xem 'val' có tồn tại trong  $A$  và vị trí đầu tiên của nó ('idx') có nhỏ hơn 'current\_limit' hay không.
  - Nếu thỏa mãn ( $idx < \text{current\_limit}$ ): Đây là nước đi hợp lệ. Cập nhật 'current\_limit = idx'. Lượt chơi chuyển sang người kia.

- Nếu không thỏa mãn: Người chơi hiện tại không thể đi tiếp và thua cuộc. Trò chơi kết thúc.
4. **Kết thúc mảng B:** Nếu duyệt hết mảng  $B$  mà chưa ai thua, người đến lượt tiếp theo sẽ thua (vì không còn phần tử  $B[i]$  để thực hiện thao tác).

### 3 Lý do lựa chọn phương pháp (Tính đúng đắn)

1. **Tính tối ưu cục bộ dẫn đến tối ưu toàn cục:** Đây là trò chơi đối kháng có tổng bằng không (zero-sum game) với thông tin hoàn hảo. Việc chọn chỉ số  $k$  nhỏ nhất giúp cực tiểu hóa không gian tìm kiếm của đối thủ. Nếu chọn một chỉ số  $k' > k$ , ta sẽ để lại một đoạn mảng dài hơn, tạo thêm cơ hội cho đối thủ. Vì đối thủ cũng chơi tối ưu, việc "nuông tay" (chọn  $k'$ ) là chiến thuật sai lầm.
2. **Cấu trúc mảng đơn điệu:** Do thao tác cắt hậu tố, các chỉ số được chọn qua từng lượt  $idx_1, idx_2, \dots$  bắt buộc phải tạo thành một dãy giảm dần nghiêm ngặt ( $idx_1 > idx_2 > \dots$ ). Việc kiểm tra `first_pos[val] < current_limit` đảm bảo tính chất này. Nếu vị trí xuất hiện đầu tiên của  $B[i]$  đã lớn hơn hoặc bằng giới hạn cắt trước đó, thì mọi vị trí xuất hiện sau đó của  $B[i]$  cũng chắc chắn không hợp lệ. Do đó, chỉ cần kiểm tra `first_pos` là đủ.

## 4 Phân tích độ phức tạp

### 4.1 Độ phức tạp thời gian (Time Complexity)

- **Tiền xử lý mảng A:** Ta duyệt mảng  $A$  một lần để xây dựng bản đồ `first_pos`. Độ phức tạp là  $O(n)$ .
- **Duyệt mảng B:** Ta duyệt mảng  $B$  một lần, các thao tác kiểm tra và cập nhật là  $O(1)$  nhờ bảng băm (hash map) hoặc mảng chỉ số. Độ phức tạp là  $O(m)$ .
- **Tổng cộng:**  $O(n + m)$ . Với  $n, m \leq 2 \times 10^5$ , thuật toán chạy rất nhanh trong giới hạn thời gian (thường là 1 giây).

### 4.2 Độ phức tạp không gian (Space Complexity)

- Mảng `first_pos`: Trong trường hợp xấu nhất cần lưu  $O(n)$  giá trị khác nhau.
- Mảng  $A$  và  $B$ :  $O(n + m)$ .
- **Tổng cộng:**  $O(n + m)$ .

## 5 Code cài đặt

```

1 import sys
2
3 # Tang giới hạn de quy hoặc buffer đọc/ghi nếu cần
4 input = sys.stdin.read
5

```

```

6 def solve():
7     # Doc du lieu nhanh
8     data = input().split()
9     if not data:
10         return
11
12     iterator = iter(data)
13     try:
14         n = int(next(iterator))
15         m = int(next(iterator))
16
17         # Doc mang A
18         A = []
19         for _ in range(n):
20             A.append(int(next(iterator)))
21
22         # Doc mang B
23         B = []
24         for _ in range(m):
25             B.append(int(next(iterator)))
26     except StopIteration:
27         return
28
29     # Tien xu ly: Luu vi tri xuat hien DAU TIEN cua moi so trong A
30     # Vi chien thuat toi uu la luon chon chi so nho nhat
31     first_pos = {}
32     for idx, val in enumerate(A):
33         # idx trong Python bat dau tu 0.
34         # Luu y: Bai toan goc dung 1-based index, nhung so sanh tuong doi
35         # giua cac index trong lap trinh 0-based van dung.
36         if val not in first_pos:
37             first_pos[val] = idx
38
39     # Gioi han hien tai (current_limit).
40     # Khoi tao la n + 1 (lon hon moi chi so co the trong A)
41     current_limit = n + 1
42
43     # Bien theo doi luot choi. 0 la Alice, 1 la Bob.
44     turn = 0
45
46     for val in B:
47         # Neu so can tim khong co trong A -> Nguoi choi hien tai thua ngay
48         lap tue
49         if val not in first_pos:
50             print("Bob" if turn == 0 else "Alice")
51             return
52
53         idx = first_pos[val]
54
55         # Kiem tra dieu kien: chi so tim duoc phai nam TRUOC diem cat cu
56         # tue la phai thuoc phan tien to chua bi xoa
57         if idx < current_limit:
58             # Hop le: Cap nhat diem cat moi va doi luot
59             current_limit = idx
60             turn = 1 - turn
61         else:
62             # Khong tim thay vi tri hop le (idx >= limit da bi xoa)
63             # Nguoi choi hien tai khong the di -> Thua

```

```

63         print("Bob" if turn == 0 else "Alice")
64         return
65
66     # Neu vong lap ket thuc ma chua ai thua (tuc la di het mang B):
67     # Luot tiep theo se danh cho nguoi 'turn', nhung mang B da het
68     # -> Ho khong the thuc hien thao tac -> Nguoi do thua.
69     print("Bob" if turn == 0 else "Alice")
70
71 if __name__ == '__main__':
72     solve()

```